TurboQuant 실전 구현: 프로덕션 LLM 추론 엔진에서의 3비트 KV 캐시 압축

AmesianX
독립 연구자
amesianx@gmail.com
https://powerhacker.net
https://github.com/AmesianX/TurboQuant

Abstract

본 논문은 Walsh-Hadamard Transform(WHT)과 Lloyd-Max 양자화 기반 KV 캐시 압축 방법인 TurboQuant을, 가장 널리 배포된 오픈소스 LLM 추론 엔진 llama.cpp에 세계 최초로 프로덕션 수준으로 구현한 결과를 제시한다. 본 구현은 KV 캐시를 요소당 3비트로 압축하여 Gemma 4 26B 모델에서 FP16 대비 **4.2배 메모리 절감**과 **6% perplexity 저하**만을 달성했다. 원본 논문의 충실한 구현을 넘어 세 가지 독창적 기여를 한다: (1) MMSE와 극치 통계학(EVT)에서 유도한 동적 어텐션 샤프닝 공식 $\alpha(N) = 1 + c\sqrt{\ln N}$; (2) flash attention 커널의 반정밀도 누적 버그 발견 및 해결 — FP16 산술의 정밀도 손실이 IWHT 버터플라이에 의해 증폭되어 MoE expert routing이 비결정적으로 동작하는 문제; (3) 5개 head dimension(64–576)에 걸친 20개 양자화 타입 변형, cross-head WHT, 비대칭 K/V 구성을 지원하는 완전한 엔지니어링 프레임워크. 전체 구현을 https://github.com/AmesianX/TurboQuant에 오픈소스로 공개한다.

1 서론

긴 컨텍스트 윈도우를 가진 대규모 언어 모델(LLM)은 키-밸류(KV) 캐시에 상당한 GPU 메모리를 요구한다. 262K 컨텍스트의 26B 파라미터 Gemma 4 모델의 경우, FP16 KV 캐시만으로 5.4 GiB를 소비하며 이는 종종 모델 가중치 자체를 초과한다. Google DeepMind가 ICLR 2026에서 발표한 TurboQuant [1]은 Walsh-Hadamard Transform(WHT) 후 Lloyd-Max 양자화를 적용하고, 선택적으로 QJL(Quantized Johnson-Lindenstrauss) 보정 항을 추가하여 KV 캐시 항목을 3–4비트로 압축한다.

이론적 프레임워크는 우아하지만, 본 연구 이전에 프로덕션 구현은 존재하지 않았다. 연구 논문과 실제 CUDA 커널 사이의 간극을 메우려면 논문이 다루지 않는 수많은 엔지니어링 문제를 해결해야 한다: WHT 버터플라이 연산을 위한 공유 메모리 레이아웃, 역변환의 정밀도 요구사항, 기존 flash attention 프레임워크와의 상호작용, 현대 아키텍처의 다양한 head dimension(64, 128, 256, 512, 576) 지원.

본 논문에서는 llama.cpp의 flash attention vector 커널(`fattn-vec`) 내 완전한 구현을 기술하고, 실제 모델에서의 성능을 보고하며, 원본 논문을 넘어서는 세 가지 기여를 제시한다:

1. **동적 어텐션 샤프닝** (§4): 양자화 노이즈가 softmax 분포를 평탄화한다. MMSE 이론과 EVT에서 컨텍스트 길이 적응형 보정 $\alpha(N) = 1 + c\sqrt{\ln N}$을 유도하여 경험적 튜닝을 제거했다.
2. **반정밀도 병리학** (§5): 업스트림 flash attention 커널의 FP16(`half2`) 누적 경로가 정밀도 손실을 야기하고, 이것이 IWHT 버터플라이에 의해 증폭되어 MoE expert routing에서 치명적 비결정성을 초래함을 발견하고 해결했다. TBQ 밸류 타입에 FP32 누적을 강제하여 perplexity가 2% 개선되었다.
3. **다중 변형 엔지니어링** (§6): 블록 크기 64–576에 걸친 20개 TBQ/TBQP 타입 변형을 구현했다. cross-head WHT($H_8 \otimes H_{64}$), 비대칭 K/V dimension(GLM 576/512), 블록별 vs. 전역 정규화, QJL 잔차 보정을 포함하며, 모두 템플릿 인스턴스화된 flash attention 프레임워크 내에서 동작한다.

2 배경

2.1 TurboQuant

TurboQuant [1]은 세 단계로 KV 캐시 벡터를 압축한다:

1. 랜덤화 WHT. d차원 벡터 $\mathbf{x}$에 대해 $\mathbf{w} = H_d \cdot (\mathbf{s} \odot \mathbf{x})$를 계산한다. H_d는 $d \times d$ Hadamard 행렬이고 $\mathbf{s} \in \{-1, +1\}^d$는 고정 랜덤 부호 패턴이다. WHT는 입력 구조와 무관하게 계수 분포를 균등화하여, Lindeberg CLT에 의해 모든 좌표가 근사적으로 i.i.d. 가우시안이 된다.

2. Lloyd-Max 양자화. 각 WHT 계수를 가우시안 분포에 최적인 Lloyd-Max 코드북으로 b비트 양자화한다. $b = 3$(8레벨)의 이론적 SQNR은 31.2, $b = 4$(16레벨)는 56.2이다.

3. QJL 보정 (선택). TurboQuant-prod(TBQP)는 $(b-1)$비트 Lloyd-Max 양자화와 잔차에 대한 1비트 QJL 투영을 결합하여, 추가적인 어텐션 스코어 분산 대가로 복원 MSE를 개선한다.

디코딩. 어텐션 시, 쿼리를 WHT 변환하여 키 도메인과 매칭한다: score $= \langle H_d(\mathbf{s} \odot \mathbf{q}),\ \mathrm{dequant}(\mathbf{k}_{\mathrm{tbq}})\rangle / d$. 밸류의 경우, WHT 도메인에서 가중합을 계산한 후 역변환한다: $\mathbf{v}_{\mathrm{out}} = \mathbf{s} \odot H_d(\sum_k \alpha_k \cdot \mathrm{dequant}(\mathbf{v}_k))/d$.

3 시스템 설계

3.1 쿼리 전처리: GPU-Side WHT

원본 TurboQuant 논문은 WHT를 양자화(CPU측)에서 수행한다고 가정한다. Flash attention 커널에서는 쿼리도 런타임에 WHT 변환해야 한다. GPU 레지스터와 공유 메모리만으로 구현했다:

$d = 256$ **(128 스레드, 각 2원소).** 부호 곱셈(레지스터) → 스테이지 7(스트라이드 128, 레지스터 버터플라이) → 스테이지 0–4(`__shfl_xor_sync` 워프 셔플) → 스테이지 5–6(공유 메모리, XOR 주소). 전역 메모리 트래픽 제로.

$d = 512$ **(128 스레드, 각 4원소).** 스테이지 8–7(스트라이드 256, 128, 레지스터) → 스테이지 0–4(워프 셔플) → 스테이지 5–6(공유 메모리). 총 9단계 버터플라이, 전역 메모리 트래픽 제로.

3.2 K 스코어링: WHT 도메인 내적

TBQ 키의 경우 내적을 WHT 도메인에서 직접 계산한다:

$$\mathrm{score} = \sum_{i=0}^{d-1} c_{\mathrm{idx}[i]} \cdot \mathrm{norm} \cdot Q_{\mathrm{wht}}[i] \tag{1}$$

TBQP 키는 QJL 투영의 두 번째 항이 추가된다:

$$\mathrm{score}_{\mathrm{tbqp}} = \mathrm{score}_{\mathrm{mse}} + d_{\mathrm{qjl}} \cdot \sum_i \mathrm{sign}_{\mathrm{qjl}}[i] \cdot Q_{\mathrm{wht2}}[i] \tag{2}$$

3.3 V 누적 및 IWHT 디코딩

밸류 벡터는 WHT 도메인에서 역양자화되고 온라인 softmax로 누적된다. KV 루프 완료 후 IWHT 역변환:

$$\mathbf{v}_{\mathrm{out}} = \frac{1}{d}\, \mathbf{s} \odot H_d\left(\frac{\sum_k e^{\alpha \cdot s_k} \cdot \mathbf{v}_k}{\sum_k e^{\alpha \cdot s_k}}\right) \tag{3}$$

3.4 블록별 vs. 전역 정규화

$d = 512$ TBQ3의 경우 WHT 벡터의 각 256원소 절반을 독립적으로 정규화한다(블록별 *norm*). 두 반쪽의 에너지가 512-point WHT 후 크게 다를 수 있기 때문이다. TBQP3는 QJL 잔차 보정이 전체 512원소 벡터에 걸쳐 동작하므로 전역 *norm*을 사용한다.

4 동적 어텐션 샤프닝

4.1 Softmax 평탄화 문제

K 캐시 항목의 양자화 노이즈가 각 어텐션 스코어 $s_i = \mathbf{q}^\top \mathbf{k}_i$에 독립 노이즈 ϵ_i를 추가한다. 관측 스코어 $\hat{s}_i = s_i + \epsilon_i$의 softmax 분포는 s_i의 분포보다 평탄하다: 노이즈가 실효 온도를 높여 올바른 최댓값에 대한 확률 질량을 감소시킨다.

4.2 $\alpha(N)$ 유도

샤프닝된 스코어의 softmax가 원본과 일치하도록 곱셈 보정 $\alpha > 1$을 구한다.

MMSE 조건. 샤프닝된 스코어의 SNR이 원본과 일치하려면:

$$\alpha \approx 1 + \frac{1}{2 \cdot \mathrm{SQNR}_{\mathrm{eff}}} \tag{4}$$

EVT 보정. N개 경쟁 토큰 중 최대 노이즈는 Gumbel 분포에 의해 $\sqrt{2 \ln N}$으로 성장한다:

$$\boxed{\alpha(N) = 1 + c \cdot \sqrt{\ln N}} \tag{5}$$

여기서 $c = 1/(2 \cdot \mathrm{SQNR}_{\mathrm{eff}} \cdot \sqrt{\ln N_0})$, 기준 $N_0 = 2048$.

4.3 타입별 계수

타입	구조	$\mathrm{SQNR}_{\mathrm{eff}}$	c	α(2K)
TBQ3	3비트 Lloyd-Max	31.2	0.00579	1.016
TBQP3	2비트 + 1비트 QJL	13.8	0.01304	1.036
TBQ4	4비트 Lloyd-Max	56.2	0.00326	1.009
TBQP4	3비트 + 1비트 QJL	24.5	0.00724	1.020

Table 1: 샤프닝 계수. TBQP 타입은 QJL 랜덤 투영이 어텐션 스코어에 두 번째 노이즈 소스를 추가하므로 실효 SQNR이 낮다.

5 Flash Attention의 반정밀도 병리학

5.1 버그

업스트림 `fattn-vec` 커널은 네이티브 FP16을 지원하는 CUDA 아키텍처에서 `V_DOT2_F32_F16_AVAILABLE`을 정의한다. 활성화 시 VKQ 누적기는 `half2`(16비트), 공유 메모리 `KQ[]`는 `half`, 쿼리 레지스터 `Q_reg`는 `half2`를 사용한다.

표준 FP16 KV 캐시에서는 이것이 정확하다. 그러나 TBQ 밸류에서는 세 가지 정밀도 손실이 복합된다:

손실 1: VKQ 누적. $\sum_k \text{softmax}_k \cdot \mathbf{v}_k$가 `half2`로 누적된다. 각 덧셈이 FP16(10비트 가수부)으로 절삭되어, 수백 개 KV 항목에 걸쳐 라운딩 오차가 누적된다.

손실 2: 공유 메모리 스테이징. reduce 단계에서 `__shared__ half KQ[]`에 부분합을 쓴 후, IWHT가 이를 `float*`로 읽는다 — 타입 불일치가 값을 추가 절삭한다.

손실 3: WHT 증폭. IWHT 버터플라이는 인접 원소의 합과 차를 계산한다. d-point WHT에서 각 출력은 $\log_2 d$ 버터플라이 스테이지를 통해 모든 d개 입력에 의존한다. 한 입력의 작은 라운딩 오차 ϵ이 출력에서 $O(\sqrt{d})$ 오차로 전파된다. $d = 512$: $\sqrt{512} \approx 22.6$배 증폭.

5.2 증상

증폭된 오차가 MoE expert routing 스코어를 결정 경계 너머로 밀어, 동일 입력과 `temp=0`에서도 실행마다 다른 expert가 선택된다. 이로 인해 토큰 위치 23–27에서부터 완전히 다른 출력 시퀀스가 생성되었다.

5.3 수정

모든 TBQ V 타입 템플릿 인스턴스에서 커널 헤더 전에 매크로를 undefine하여 FP32 누적을 강제한다:

```
#include "../common.cuh"
#undef V_DOT2_F32_F16_AVAILABLE
#include "../fattn-vec.cuh"
```

추가로, IWHT 입력을 위한 float 레지스터 스테이징을 구현했다:

```
float tbq_regs[D_V / nthreads];
// reduce 루프가 tbq_regs에 저장
__syncthreads();
float *buf = (float*)&KQ[0];
buf[i0 + tid] = tbq_regs[i0/nthreads];
```

5.4 영향

구성	PPL	vs FP16
FP16/FP16 기준선	419.8	1.00×
half2 경로 (수정 전)	454.7	1.08×
float 경로 (수정 후)	**445.7**	**1.06×**

Table 2: FP32 누적 강제에 의한 perplexity 개선.

비결정성 완전 해소: FP16 기준선은 20회 연속 동일 출력, TBQ도 이제 이 동작과 일치한다.

6 엔지니어링 프레임워크

6.1 타입 시스템

블록 크기별로 조직된 20개 양자화 타입을 정의한다:

접미사	블록 크기	Head Dim	비고
_0	256	256, 512	주력 타입
_1	128	128	표준 트랜스포머
_2	64	64	소형 head 모델
_3	64	64	Cross-head WHT
_4	576	576	GLM 분할 256+256+64

Table 3: TBQ 블록 크기 변형. 각각 TBQ3/TBQ4/TBQP3/TBQP4 하위 타입이 있다.

6.2 Cross-Head WHT (_3 타입)

$d = 64$ head dimension 모델에서는 64-point WHT의 탈상관이 불충분하다. Kronecker 구조 $H_{512} = H_8 \otimes H_{64}$를 이용한 cross-head 변형을 구현했다: 8개 head가 하나의 양자화된 KV 블록을 공유하고, H_8 부호를 스코어링 시 적용:

$$\text{score}_h = \frac{1}{8} \sum_{g=0}^{7} H_8[h, g] \cdot \langle \mathbf{q}_h, \text{dequant}(\mathbf{k}_g) \rangle \tag{6}$$

동작을 위해 두 가지 핵심 수정이 필요했다:

Per-head 정규화. 초기 구현은 512개 WHT 계수 전체에 대한 global L2 norm을 사용했다. 한 head의 에너지가 지배적이면 나머지 head는 부적절한 스케일로 양자화되어 정보를 잃는다. $d = 512$ per-block norm과 동일한 원리로 per-head RMS 정규화로 전환했다.

$d = 64$에서 K의 QJL 제거. TBQP 타입은 $(b-1)$비트 Lloyd-Max + 1비트 QJL을 사용한다. $d = 64$에서 QJL의 랜덤 프로젝션은 어텐션 스코어에 올바른 토큰과 차점자 사이의 signal gap을 초과하는 노이즈를 추가한다. 순수 3비트 Lloyd-Max(TBQ3, 8 centroid)는 TBQP3(2비트 +QJL) 대비 2.3× 높은 스코어 SQNR을 달성한다. $d = 64$에서 K를 자동으로 TBQP→TBQ로 전환하여 모든 비트를 centroid 정밀도에 집중한다.

6.3 MMSE 동적 소프트닝 ($d = 64$)

$d \geq 256$에서는 양자화 노이즈가 어텐션 gap 대비 작아 샤프닝($\alpha > 1$)으로 softmax 평탄화를 보정한다. $d = 64$에서는 노이즈가 토큰 순위 자체를 뒤집을 만큼 크다. 샤프닝은 잘못된 토큰에 대한 확신을 증폭시켜 반복 루프를 유발한다.

MMSE 최적 소프트닝 보정을 유도했다:

$$\alpha(N) = \frac{\text{SQNR}}{\text{SQNR} + \sqrt{\ln N / \ln N_0}} \tag{7}$$

$\alpha < 1$: softmax를 더 평탄하게 만들어 과확신을 줄인다. EVT 항 $\sqrt{\ln N / \ln N_0}$는 긴 컨텍스트에서 더 강한 소프트닝을 적용한다.

동일한 MMSE/EVT 이론에서 높은 SQNR($d \geq 256$)에서는 샤프닝, 낮은 SQNR($d = 64$)에서는 소프트닝이 도출되는 통합 프레임워크를 형성한다.

6.4 GLM 576: 비대칭 K/V

GLM-4는 MLA로 $d_K = 576, d_V = 512$를 사용한다. `D_V` 템플릿 파라미터로 이를 지원하며, 576차원 K는 분할 WHT(256+256+64), 512차원 V는 표준 512-point WHT를 사용한다.

6.5 SWA F16 바이패스

Gemma 4는 35개 레이어 중 25개에 Sliding Window Attention(SWA)을 사용한다. SWA 캐시는 300 MiB에 불과하지만(전역 KV 5120 MiB 대비), 최근 고관련성 토큰에 어텐드하므로 출력 품질을 지배한다. SWA K/V를 자동으로 FP16 업그레이드하여 가장 중요한 곳의 양자화 오차를 완전히 제거했다.

6.6 V Rotation 비호환성

llama.cpp KV 캐시는 기본적으로 K와 V에 RoPE rotation을 적용한다. K는 안전하다: $\mathbf{q}^\top R^\top R\mathbf{k} = \mathbf{q}^\top\mathbf{k}$ (직교 변환의 성질로 내적에서 상쇄). 그러나 V는 IWHT 디코드에 대응하는 역rotation이 없어, 출력이 회전된 좌표계에 있게 된다. 모든 TBQ 구성에서 V rotation을 비활성화했다 (`attn_rot_v=false`).

7 개발 타임라인

구현은 8일에 걸쳐 5개 주요 세션으로 진행되었으며, 각 세션은 고유한 엔지니어링 과제를 해결했다:

v1.4.0 (4/1–3): 기반 + GLM 576. head_dim=256(Qwen, LLaMA)과 128(표준 트랜스포머)을 위한 초기 CUDA 커널 구현. GLM-4.7-Flash의 head_dim=576은 2의 거듭제곱이 아니어서 분할 WHT를 설계: $576 = 256 + 256 + 64$. 500회 시뮬레이션으로 full QR 직교 회전 대비 −3.4% MSE, 메모리 오버헤드 0% 확인.

v1.5.0 (4/4): 업스트림 리베이스 + Gemma 4. 업스트림 llama.cpp에 완전한 3-way merge 리베이스(26개 충돌 해결). Gemma 4: hybrid SWA(global head_dim=512 10레이어 + SWA head_dim=256 50레이어), 가변 GQA(16/4). SWA 캐시 타입 자동 재매핑, 가변 GQA 우회, 512-point 단일 패스 WHT(9단계 버터플라이, 128스레드×4원소) 구현.

v1.5.1 (4/4 PM): SWA F16 바이패스. SWA 캐시(300 MiB)가 25레이어를 통해 출력 품질을 지배한다는 발견. SWA K/V를 FP16으로 자동 업그레이드하여 수학 벤치에서 사상 첫 f16 동급 달성(37.4/65 vs f16 36.6/65).

v1.5.2 세션 1 (4/7): 어텐션 샤프닝. PPL 21% 격차의 원인이 비트 수가 아니라 softmax 평탄화임을 규명. 서브블록 스케일 실험 실패(WHT가 이미 분산 균등화). MMSE 이론에서 $\alpha = 1 + 1/(2 \times \mathrm{SQNR})$ 유도. V rotation 버그 발견. PPL: 509.9 → 454.7.

v1.5.2 세션 2 (4/7–8): 정밀도 수정. temp=0 비결정적 출력 → CUDA 그래프, 풀 할당, threadfence, MoE MUL_MAT_ID 등 모두 원인 아님으로 제거. 근본 원인: 업스트림 `fattn-vec`의 `half2` 누적이 IWHT 버터플라이에 의해 증폭. 70개 TBQ V 템플릿 인스턴스에 FP32 강제. PPL: 454.7 → 445.7. 동적 $\alpha(N)$ 공식 구현.

8 실험 결과

모든 실험은 NVIDIA DGX Spark(GB10, 128GB VRAM), CUDA 12.8, Gemma 4 26B-A4B-it(MoE, UD-Q4_K_XL)에서 262K 컨텍스트로 수행.

8.1 Perplexity

구성	PPL (wiki-2K)	vs FP16
FP16/FP16	419.8	1.00×
v1.5.1 (샤프닝 전)	509.9	1.21×
+ 블록별 norm	498.2	1.19×
+ 어텐션 샤프닝	455.0	1.08×
+ V rotation 수정	448.1	1.07×
+ 동적 α	454.7	1.08×
+ FP32 누적	**445.7**	**1.06×**

Table 4: v1.5.1에서 v1.5.2까지의 점진적 PPL 개선.

8.2 수학 정확도

35문제 수학 벤치마크(2×2/3×3 행렬곱, 스칼라 연산, 0–2000 토큰 필러)에서 temp=0으로 평가:

구성	평균 (/35)	피크	10회 결과
FP16	20.1	21	19–21
TBQP3/TBQ3	**19.1**	**23**	16–23

Table 5: 수학 벤치 결과. TBQ 피크(23)가 FP16 최고(21)를 초과.

8.3 메모리

구성	Global KV	총 KV	압축
FP16/FP16	5,120 MiB	5,420 MiB	1.0×
TBQP3/TBQ3	990 MiB	1,290 MiB	4.2×

Table 6: 262K 컨텍스트에서의 KV 캐시 메모리 (Gemma 4 26B MoE).

8.4 결정성

FP32 누적 수정 후 temp=0으로 20회 연속 실행 시, 3비트 양자화 경계의 1–2개 보더라인 토큰을 제외하고 동일한 출력을 생성한다. FP16 기준선은 20/20 동일. 수정 전에는 토큰 23에서부터 극단적 발산이 발생했다.

9 관련 연구

KV 캐시 양자화. KIVI [4]와 KVQuant [5]는 변환 없이 채널별/토큰별 양자화로 KV 캐시를 압축한다. TurboQuant의 WHT 전처리는 입력 구조와 무관하게 계수 분포를 균등화하여 최적 Lloyd-Max 양자화를 가능하게 한다.

Flash Attention. FlashAttention [2, 3]과 변형들은 타일링과 온라인 softmax로 어텐션 계산을 최적화한다. 본 구현은 단일 토큰 디코딩에 특화된 llama.cpp `fattn-vec` 커널을 확장한다.

어텐션 스코어 보정. SmoothQuant [7] 등 outlier-aware 방법은 활성화 양자화를 다루지만 KV 양자화 노이즈에 의한 softmax 평탄화는 보정하지 않는다. 본 연구의 동적 샤프닝은 양자화 SQNR에서 유도된 최초의 원리적 보정이다.

10 결론

TurboQuant KV 캐시 압축의 최초 프로덕션 구현을 제시하여, 최신 MoE 모델에서 4.2배 메모리 절감과 6% perplexity 비용을 달성했다. 동적 어텐션 샤프닝 공식은 양자화 유도 softmax 평탄화에 대한 이론적 근거가 있는, 하이퍼파라미터 없는 보정을 제공한다. 반정밀도 증폭 병리학의 발견은 WHT 기반 방법의 수치 정밀도 요구사항이 표준 양자화와 근본적으로 다름을 강조한다 — 원본 논문에서 부재한 고려사항으로, 향후 WHT 기반 압축 연구에 관련될 수 있다.

5개 head dimension에 걸친 20개 양자화 변형을 지원하는 전체 구현을 https://github.com/AmesianX/TurboQuant에 llama.cpp 포크로 공개한다.

감사의 글

본 연구는 NVIDIA DGX Spark에서 독립적으로 수행되었다. llama.cpp 커뮤니티와 이론적 기반을 제공한 Google DeepMind의 TurboQuant 저자들에게 감사한다.

References

[1] S. Ashkboos, A. Mohtashami, R. Pramanik, Y. Li, A. Jaggi, D. Alistarh. TurboQuant: Online KV Cache Quantization via Hadamard Transform. In *Proceedings of ICLR*, 2026.

[2] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.

[3] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *ICLR*, 2024.

[4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *ICML*, 2024.

[5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[6] A. Zandieh, M. Daliri, I. Han. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead. In *NeurIPS*, 2024.

[7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *ICML*, 2023.